

MODULE 03

SYLLABUS: Memory management - Address Space, Memory API, Address Translation - An Example, Dynamic (Hardware-based) Relocation, Segmentation: Generalized Base/Bounds, Address translation in segmentation, Support for Sharing. Virtual memory - Paging: Introduction, page tables and hardware support, TLBs, Example: Accessing An Array, - TLB hits and misses, Handling TLB misses, TLB structure, Reducing the page table size. Going beyond physical memory - Swap space, page fault and its control flow, page replacement policies, Thrashing.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR
SCAN QR CODE](#)



MEMORY MANAGEMENT

A **process** is a running program that needs memory to hold its code, data, and runtime information. The CPU can only work with data present in main memory (RAM). When multiple processes run simultaneously, they must safely share this limited physical memory — transparently to each process. This is what memory management solves.

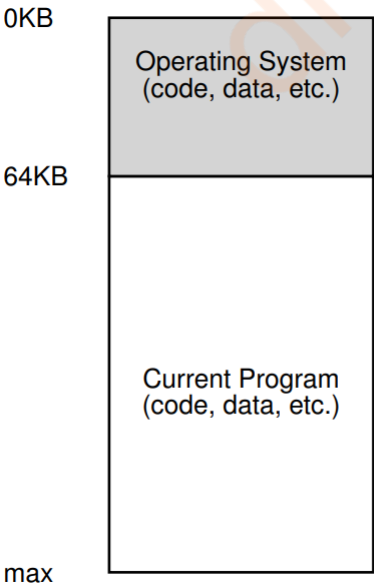
Evolution of Memory Usage

Early Systems: Only one program ran at a time. The OS loaded at address 0, the program occupied the rest. No memory abstraction — the program accessed physical memory directly. Simple but wasteful; CPU sat idle during I/O.

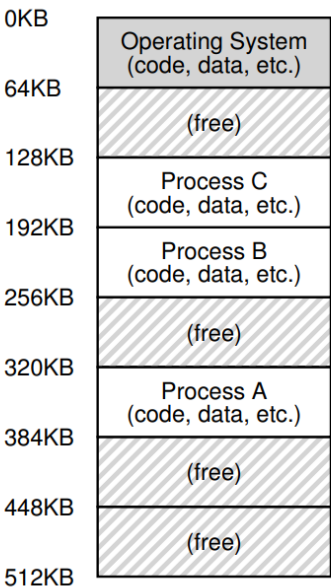
Multiprogramming and Time Sharing: Multiple processes loaded simultaneously; CPU switches between them. Two critical needs arose:

- **Protection** — one process must not access or corrupt another's memory.
- **Isolation** — each process must operate independently, unaware of others.

These needs directly motivated virtual memory and address spaces.



Operating Systems: The Early Days



Multiprogramming and Time Sharing

The Address Space

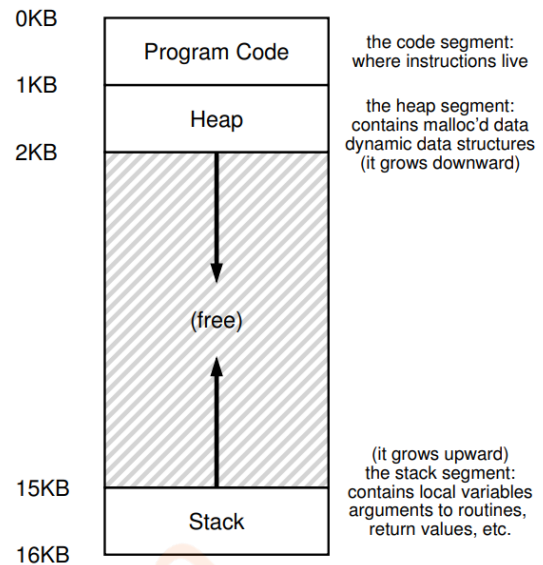
An **address space** is the complete set of memory addresses a process believes it owns exclusively. It is an OS abstraction giving each process the illusion of having its own large, private, contiguous memory — even though physical memory is shared.

Components of an Address Space

Code Segment — Stores compiled instructions. Static (does not change at runtime). Placed at low addresses. Easy to place and manage since it doesn't grow.

Heap — Dynamic memory allocation (size unknown at compile time). Managed by the programmer via `malloc()` and `free()`. Grows **downward** (toward higher addresses) as more memory is allocated.

Stack — Tracks function calls, local variables, parameters, return values. Managed automatically by the compiler. Grows **upward** (toward lower addresses). When a function returns, its frame is discarded — efficient but short-lived.



Memory Virtualization

Memory virtualization is the OS technique of giving each process the illusion of having its own large, private, contiguous memory space, even though physical memory is limited and shared.

The running program believes it is loaded at address 0 and has the entire address space to itself. In reality it may be anywhere in physical memory, sharing RAM with many other processes.

Goals of Virtual Memory — three fundamentals:

Transparency — The process must be completely unaware memory is virtualized. It behaves as if it has its own private physical memory. The OS and hardware maintain this illusion invisibly.

Efficiency — Virtualization must not slow programs significantly (time) nor waste memory on management structures (space). Hardware support — especially TLBs — is essential.

Protection — Processes must be isolated from each other and the OS. A process performing a load, store, or fetch must not access memory outside its own address space. This is **isolation** — if two entities are properly isolated, one can fail without affecting the other.

Memory API

The **Memory API** is the set of library functions and system calls allowing programs to request, use, and release memory at runtime.

Memory Types in C

Type	Managed By	Allocation	Lifetime
Stack	Compiler (automatic)	On function call	Until function returns
Heap	Programmer (manual)	Explicit <code>malloc()</code> call	Until explicitly <code>free()</code> d

Key Memory Functions

1. `malloc()` — Allocates a block on the heap of the requested size. Returns a pointer to the new space, or `NULL` if allocation fails. Always use `sizeof()` to request the correct size. Syntax: `void* malloc(size_t size);`
Example: `int *x = (int*) malloc(sizeof(int));`

2. **free()** — Releases heap memory previously allocated. Without `free()`, memory stays occupied after it is no longer needed — causing a **memory leak**. Syntax: `free(pointer)`;
3. **calloc()** — Like `malloc()` but also zeroes out the allocated memory before returning. Prevents errors from assuming uninitialized memory is zero.
4. **realloc()** — Resizes a previously allocated block (e.g., expanding an array). Allocates a new larger region, copies old data into it, returns pointer to the new region.

Common Memory Errors

- **Forgetting to allocate memory** — Using a pointer before allocating memory for it → Segmentation fault.
- **Buffer overflow** — Writing more data into an array than it can hold → Corrupts adjacent memory.
- **Memory leak** — Allocating with `malloc()` but never calling `free()` → Program gradually consumes all memory.
- **Dangling pointer** — Freeing memory then continuing to use the pointer → Points to freed (invalid) memory.
- **Double free** — Calling `free()` on the same block more than once → Undefined behavior or crash.

Underlying OS Support

`malloc()` and `free()` are **library calls, not system calls**. They are user-level functions built on top of OS system calls: `brk()`, `sbrk()`, and `mmap()`. The OS provides raw memory management; the C library abstracts this into the convenient `malloc/free` interface.

Address Translation

Address translation is the hardware mechanism by which every virtual address generated by a process is automatically converted to the corresponding physical address where the data actually resides.

Every memory access — instruction fetch, load, or store — involves this translation. It is performed by a hardware unit called the **Memory Management Unit (MMU)**, which sits between the CPU and main memory.

Why Translation Is Needed

A process is compiled assuming its address space starts at 0. But the OS may place it anywhere in physical memory. Without translation, address 128 in the program would mean physical address 128 — which may belong to the OS or another process. Translation resolves this mismatch automatically and invisibly.

An Example of Address Translation

Consider: `x = x + 3`; with `x` at virtual address 15 KB.

Compiled instructions:

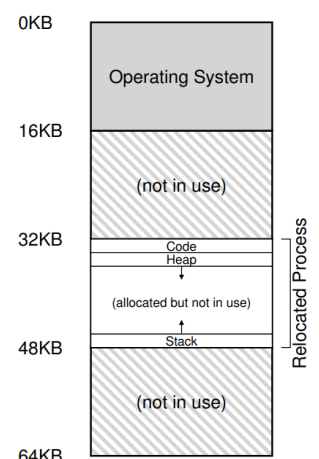
```
128: movl 0x0(%ebx), %eax    ;load 0+ebx into eax
132: addl $0x03, %eax        ;add 3 to eax register
135: movl %eax, 0x0(%ebx)    ;store eax back to mem
```

Process address space = 16 KB. OS places it at physical address **32 KB (32768)**.

MMU translates every access:

- Fetch at VA 128 → PA = 32768 + 128 = **32896**
- Load from VA 15 KB (15360) → PA = 32768 + 15360 = **48128**
- Store to VA 15 KB → PA = **48128**

The process never knows it is at 32 KB. This is **transparent address translation**.



Dynamic (Hardware-Based) Relocation

Dynamic relocation is the technique of relocating a process's address space to any physical memory location at runtime, using hardware registers (base and bounds), so the process can be moved even while running.

Base and Bounds Registers

Each CPU contains two special hardware registers per process:

Base Register — Holds the starting physical address where the process's address space is loaded. Every virtual address is added to this value to get the physical address.

Bounds (Limit) Register — Holds the size of the address space. Every virtual address is checked against this to ensure it is within legal range.

Translation formula: Physical Address = Virtual Address + Base

Safety check before use:

If Virtual Address $\geq$ Bounds $\rightarrow$ Exception (Segmentation Fault / Process Terminated)

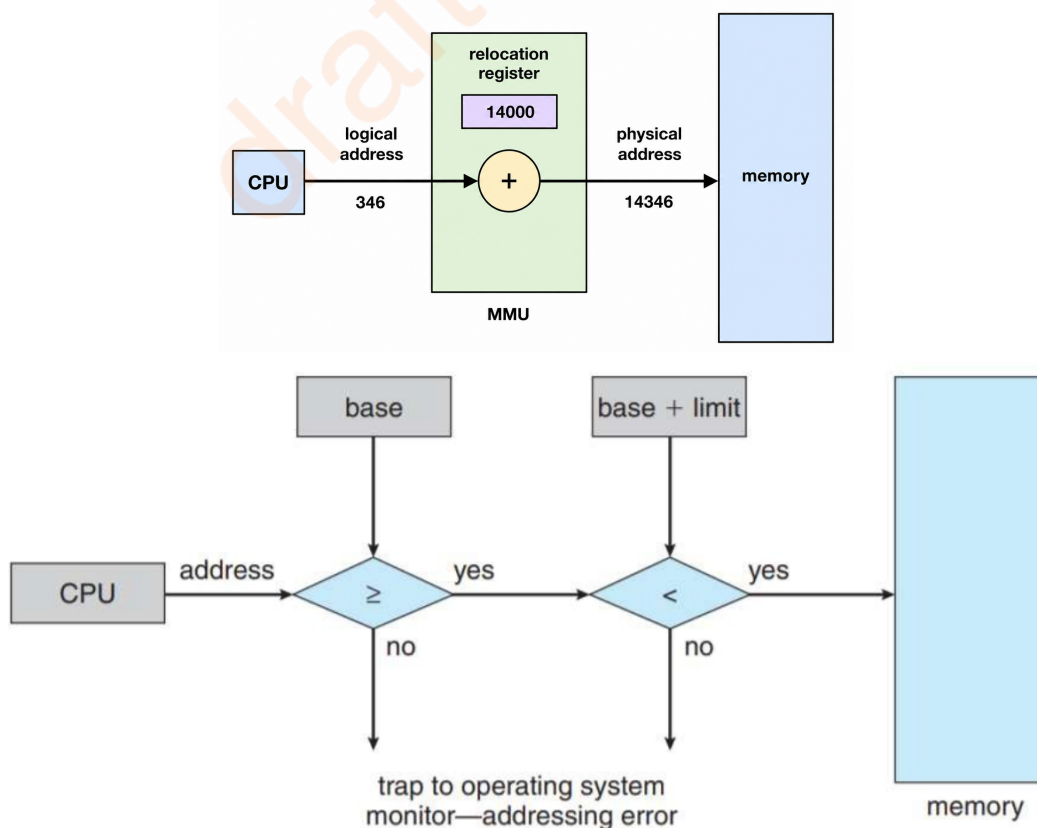
This pair forms the core of the MMU. Together they:

1. Allow placing the process anywhere in physical memory (via base).
2. Prevent the process from accessing memory outside its space (via bounds).

Example: Process address space = 16 KB. OS loads it at physical address 32 KB.

- Base register = 32768 (32 KB)
- Bounds register = 16384 (16 KB)

Virtual Address	Calculation	Result
128	$128 + 32768$	32896 ✓ ($128 < 16384$)
15360	$15360 + 32768$	48128 ✓ ($15360 < 16384$)
20000	—	Exception! ($20000 \geq 16384$)



Logical vs. Physical Address

Parameter	Logical (Virtual) Address	Physical Address
Generated by	CPU	MMU (computed)
Visible to user?	Yes	No
Also called	Virtual address	Real address
Can be changed?	Yes (relative)	Fixed in memory
Address space	Set of all logical addresses of a program	Set of all corresponding physical addresses

The MMU performs the mapping between the two.

Address Binding

Address binding is the process of mapping symbolic/logical addresses to actual physical addresses. Addresses in source code are symbolic (variable names). The compiler converts them to logical (relocatable) addresses. At runtime, the MMU maps logical to physical. The base register used here is also called the **relocation register**.

Internal Fragmentation

With base-and-bounds, the entire address space (including the unused region between heap and stack) is allocated as one contiguous block in physical memory. The unused space between heap and stack is wasted physical memory. This wasted space **inside** an allocated block is called **internal fragmentation**.

Segmentation

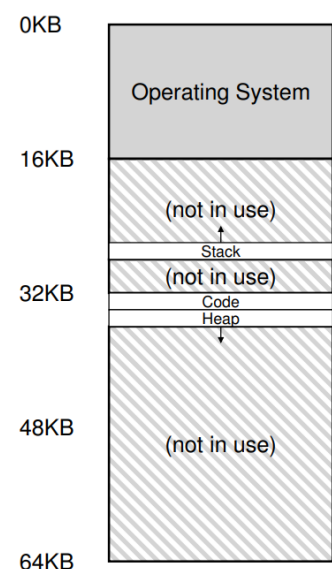
Segmentation is a memory management scheme where each logical section (segment) of a process's address space — code, heap, stack — is placed independently in physical memory, with its own base and bounds pair.

Why Segmentation?

With a single base-and-bounds pair, the entire address space (including the large unused gap between heap and stack) must be placed **contiguously** in physical memory. This wastes physical memory on space the process hasn't even used yet.

Segmentation solves this by giving each logical region its own independent placement in physical memory. Instead of one base-bounds pair for the whole process, the MMU now maintains **one base-bounds pair per segment**:

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K



Each segment is placed independently. The free region between heap and stack in the virtual address space consumes **no physical memory** at all.

Address Translation in Segmentation

To translate a virtual address to a physical address in a segmented system:

Step 1: Identify which segment the virtual address belongs to.

Step 2: Calculate offset = Virtual Address – Segment Start (start of that segment in virtual space).

Step 3: Check: offset < segment size (bounds check). If not → segmentation fault.

Step 4: Physical Address = Segment Base + Offset.

Example 1 — Code Segment: Virtual address = 100 (code starts at virtual 0 KB)

- Offset = $100 - 0 = 100$
- Bounds check: $100 < 2048$ ✓
- Physical Address = $32 \text{ KB} + 100 = 32768 + 100 = \mathbf{32868}$

Example 2 — Heap Segment: Virtual address = 4200 (heap starts at virtual 4 KB = 4096)

- Offset = $4200 - 4096 = 104$
- Bounds check: $104 < 2048$ ✓
- Physical Address = $34 \text{ KB} + 104 = 34816 + 104 = \mathbf{34920}$

Example 3 — Illegal Access: Virtual address = 7 KB (heap ends at 6 KB)

- Offset = $7168 - 4096 = 3072$
- Bounds check: $3072 > 2048 \rightarrow \mathbf{\text{Segmentation Fault!}}$

Model Question

Translate the logical address 0001010010111010 into its corresponding physical address under a segmentation system with a maximum segment size of 1024, using a segment table where the base address of each segment is determined as: $22 + 4,096 \times \text{segment number}$. (5 MARKS)

Solution

Maximum segment size = $1024 = 2^{10}$

Therefore:

- Offset requires 10 bits
- Remaining upper bits represent segment number

Logical address: 0001010010111010

Step 1 — Separate segment number and offset

Segment number bits = upper 6 bits = $000101_2 = 5$

Offset bits = lower 10 bits = 0010111010_2

Offset value: $0010111010_2 = 186$

Step 2 — Calculate segment base

base = $22 + (4096 \times \text{segment number})$

$$\begin{aligned} &= 22 + (4096 \times 5) \\ &= 22 + 20480 \\ &= 20502 \end{aligned}$$

Step 3 — Calculate physical address

Physical Address = Base + Offset

$$\begin{aligned} &= 20502 + 186 \\ &= 20688 \end{aligned}$$

Therefore, Physical Address = 20688

Segmentation Fault

When a process tries to access a virtual address that falls outside the valid bounds of any segment, the hardware raises a trap to the OS. The OS typically terminates the process. This is called a **segmentation violation** or **segmentation fault** — a term very familiar to C programmers.

Which Segment Does a Virtual Address Belong To? (Explicit Approach)

The hardware must determine which segment a virtual address belongs to before it can translate it. The most common method is the **explicit approach**: the **top few bits** of the virtual address encode the segment number, and the remaining bits are the offset.

For a 14-bit virtual address with 3 segments (needing 2 bits to identify the segment):

- Bits 13–12 : Segment identifier (00 = code, 01 = heap, 11 = stack)
- Bits 11–0 : Offset within that segment

Example: Virtual address 4200 in binary (14-bit): 01 000001101000

- Top 2 bits = **01** → Heap segment
- Remaining 12 bits = 000001101000 = **104** → offset into heap
- Physical address = Heap Base + 104 = 34 KB + 104 = **34920** ✓

The hardware reads the top bits, selects the corresponding segment's base and bounds from the segment registers, applies the bounds check on the offset, and computes the physical address.

Generalized Base/Bounds — Segment Table

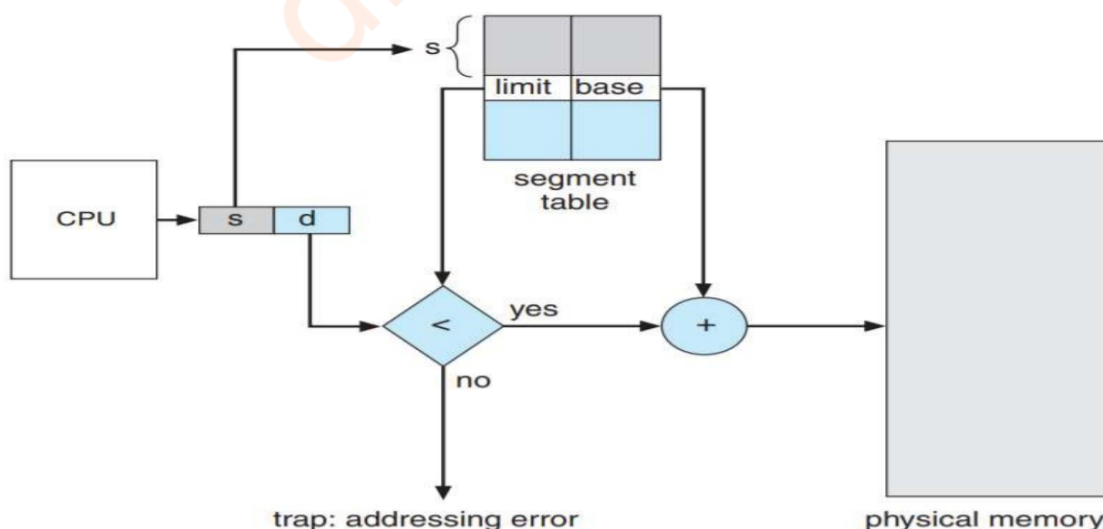
In a fully generalized segmentation system, each process has a **segment table** in memory. Each entry contains:

- **Segment Base** — starting physical address of that segment
- **Segment Limit** — length of the segment

A logical address is expressed as a pair: $\langle \text{segment-number } s, \text{offset } d \rangle$

Translation steps:

1. Use s to index into the segment table → get base and limit.
2. Check: if $d < \text{limit}$ → valid; else → trap (addressing error).
3. Physical address = base + d .



Worked Example — Segment Table:

Segment	Base	Limit
0	1400	1000
1	6300	400
2	4300	400
3	3200	1100
4	4700	1000

Query	Check	Physical Address
Seg 0, offset 430	$430 < 1000$ ✓	$1400 + 430 = \mathbf{1830}$
Seg 1, offset 10	$10 < 400$ ✓	$6300 + 10 = \mathbf{6310}$
Seg 2, offset 500	$500 < 400?$ ✗	Illegal — trap to OS
Seg 3, offset 400	$400 < 1100$ ✓	$3200 + 400 = \mathbf{3600}$
Seg 4, offset 112	$112 < 1000$ ✓	$4700 + 112 = \mathbf{4812}$

Support for Sharing in Segmentation

Segmentation naturally enables **memory sharing** between processes. Since each segment has its own base-bounds pair and is independently placed, multiple processes can have their segment registers point to the **same physical memory region**.

The most common use case is **code sharing**: if two processes run the same program, their code segments can point to the same physical pages. Since code is read-only, this is perfectly safe.

Protection Bits — each segment entry includes protection bits specifying permitted access:

Segment	Base	Size	Grows Positive?	Protection
Code	32 KB	2 KB	Yes	Read-Execute
Heap	34 KB	2 KB	Yes	Read-Write
Stack	28 KB	2 KB	No	Read-Write

- **Code** is marked Read-Execute — can be read and executed, but not written. This allows the same physical code segment to be safely shared across multiple processes.
- **Heap and Stack** are Read-Write — they hold process-specific data and must not be shared.

If a process attempts to write to a read-only segment or execute from a non-executable segment, the hardware raises a protection exception and the OS terminates the process.

Key insight: By making the code segment read-only and sharing it across virtual address spaces, the OS saves physical memory while still preserving the illusion of isolation for each process.

Model Question

A computer provides each process with 65,536 bytes of address space divided into pages of 4096 bytes each. A particular process has a text size of 32,768 bytes, a data size of 16,386 bytes, and a stack size of 15,870 bytes. Will this process fit in the machine's address space? Suppose that instead of 4096 bytes, the page size was 512 bytes, would it then fit? Each page must contain either text, data, or stack, not a mixture of two or three of them. (4 MARKS)

Solution

Case 1 — Page size = 4096 bytes

Text segment: $32768 / 4096 = 8$ pages exactly

Data segment: $16386 / 4096 = 4.0005 \Rightarrow$ Requires 5 pages

Stack segment: $15870 / 4096 = 3.87 \Rightarrow$ Requires 4 pages

Total pages required: $8 + 5 + 4 = 17$ pages

Total memory needed: $17 \times 4096 = 69632$ bytes

Hence, the process fits if address space supports at least 69,632 bytes.

Case 2 — Page size = 512 bytes

Text: $32768 / 512 = 64$ pages

Data: $16386 / 512 = 32.00+ \Rightarrow 33$ pages

Stack: $15870 / 512 = 30.99 \Rightarrow 31$ pages

Total pages: $64 + 33 + 31 = 128$ pages

Total memory: $128 \times 512 = 65536$ bytes

Hence, with smaller page size, internal fragmentation reduces and memory usage becomes more efficient.

Disadvantage of Segmentation — External Fragmentation

While segmentation is a significant improvement over a single base-bounds pair, it has one major drawback:

External Fragmentation — Over time, as segments are allocated and freed, physical memory develops scattered holes of free space. Even if total free memory is sufficient for a new segment, no single contiguous free block may be large enough.

Example: Physical memory has three free holes of 10 KB, 8 KB, and 6 KB (total 24 KB). A new 20 KB segment request will **fail** — because no single hole is 20 KB, even though 24 KB is free.

Comparison of fragmentation types:

Problem	Cause	Solution
Internal Fragmentation	Unused space inside an allocated block (base-bounds)	Segmentation (finer granularity)
External Fragmentation	Scattered free holes between variable-size segments	Paging (fixed-size blocks)

This limitation of segmentation motivates **paging**..

VIRTUAL MEMORY

Before studying virtual memory, understand why it exists. Every process needs its own memory space. Physical RAM is limited, yet modern systems run many processes simultaneously. Virtual memory solves this by giving every process the illusion of a large, private address space, even when physical RAM is insufficient.

Key terms before proceeding:

Term	Meaning
Physical Address	Actual hardware location in RAM
Virtual (Logical) Address	Address used by a process; must be translated
Page	Fixed-size block of virtual memory
Frame	Fixed-size block of physical memory (same size as a page)
Page Table	Data structure mapping pages → frames

Paging

Paging is a memory management scheme that permits the physical address space of a process to be **non-contiguous**. The virtual address space is divided into fixed-size units called **pages**, and physical memory is divided into equal-sized units called **frames**. Any free frame can hold any page, regardless of physical location.

Paging **completely eliminates external fragmentation** (no scattered unusable gaps), though a small amount of **internal fragmentation** may occur — the last page of a process may not be fully used.

How Address Space is Divided

Consider a 64-byte address space with 16-byte pages → 4 pages (Page 0 to Page 3). Physical memory has 8 frames. The pages land in scattered frames — this is the whole point of paging. The page table records which page went to which frame.

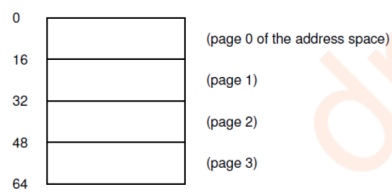


Figure 18.1: A Simple 64-byte Address Space

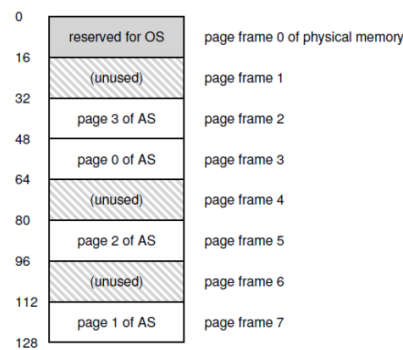
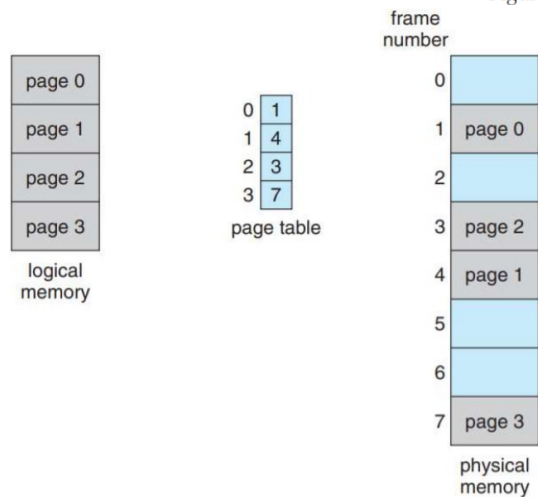


Figure 18.2: 64-Byte Address Space Placed In Physical Memory



Address Translation — How It Works

Every virtual address generated by the CPU is split into two parts:



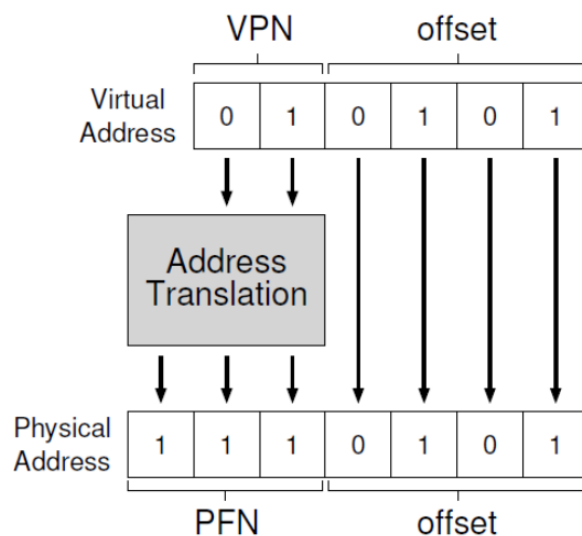
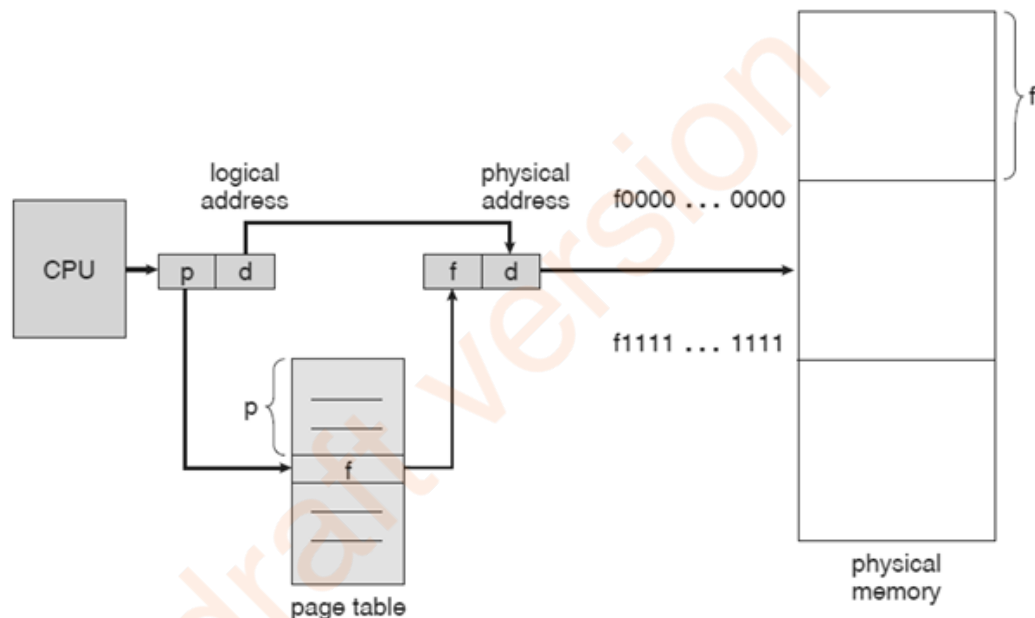
- **p (Page Number / VPN)** — index into the page table to find the physical frame number.
- **d (Offset)** — exact byte position within that page; copied unchanged to the physical address.

The hardware uses the VPN to look up the **Page Frame Number (PFN)** from the page table, then combines it with the offset:

$$\text{Physical Address} = [\text{PFN} \mid \text{Offset}]$$

Address bit formula: If logical address space = 2^m bytes and page size = 2^n bytes, then:

- Page number field = **($m - n$) bits** (high-order bits)
- Offset field = **n bits** (low-order bits)



Worked Example

Given: $n = 2$ (offset bits), $m = 4$ (total bits) $\rightarrow$ Page size = $2^2 = 4$ bytes, Physical memory = 32 bytes = 8 frames

Page table: Page 0 $\rightarrow$ Frame 5, Page 1 $\rightarrow$ Frame 6, Page 2 $\rightarrow$ Frame 1, Page 3 $\rightarrow$ Frame 2

How to solve: Convert logical address to binary $\rightarrow$ extract top $(m-n)$ bits as page number, bottom n bits as offset $\rightarrow$ look up frame $\rightarrow$ Physical Address = frame $\times$ page size + offset.

Logical Address	Page No. (p)	Offset (d)	Frame (f)	Physical Address = $f \times 4 + d$
0	0	0	5	$5 \times 4 + 0 = 20$
3	0	3	5	$5 \times 4 + 3 = 23$
4	1	0	6	$6 \times 4 + 0 = 24$
13	3	1	2	$2 \times 4 + 1 = 9$

Page Table Entry (PTE) — What's Inside

Each entry in the page table is not just a frame number. It contains several important bits:

Bit	Purpose
Valid / Present bit	1 = page is in RAM; 0 = page is on disk (triggers page fault)
Protection bits	Read / Write / Execute permissions
Dirty bit	1 = page was modified since loaded; needs to be written back to disk
Reference bit (Accessed bit)	1 = page was recently accessed; used by replacement algorithms
PFN	The actual physical frame number

The **valid bit** is critical for sparse address spaces. Unused virtual pages (e.g., the gap between heap and stack) are marked invalid — no physical frame is wasted for them.

Where Are Page Tables Stored?

Page tables can be very large. For a 32-bit address space with 4 KB pages:

- VPN = 20 bits $\rightarrow 2^{20} \approx 1$ million entries per process
- At 4 bytes per entry $\rightarrow 4$ MB per process just for the page table

This is too large to store on-chip (in the MMU). So the OS stores each process's page table in **physical memory**. A register called the **Page Table Base Register (PTBR)** holds the starting address of the current process's page table.

The consequence: Every memory access now requires **two memory accesses** — one to fetch the PTE from the page table, and one to fetch the actual data. This doubles access time — which is exactly why the TLB exists.

Translation Lookaside Buffer (TLB)

A **Translation Lookaside Buffer (TLB)** is a small, high-speed hardware cache inside the MMU that stores recently used virtual-to-physical address translations. It avoids the overhead of accessing the page table in main memory on every memory reference.

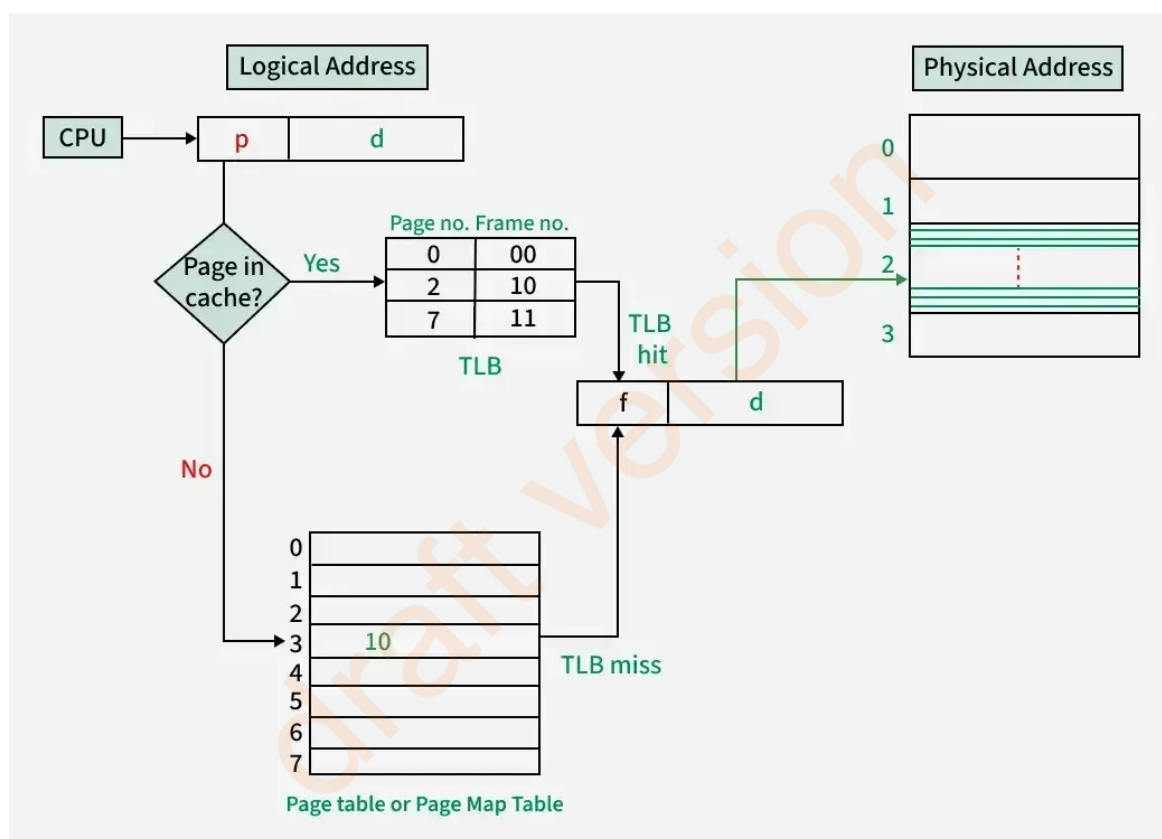
TLB Structure and Working

Each TLB entry holds: [VPN | PFN | Valid bit | Protection bits]

When the CPU generates a virtual address, the hardware extracts the VPN and checks the TLB simultaneously against all entries (**associative lookup** — all entries checked in parallel):

TLB Hit: VPN found in TLB → PFN returned immediately → physical address formed → data fetched. Very fast, no extra memory access needed.

TLB Miss: VPN not in TLB → hardware consults the page table in memory → PFN found → loaded into TLB → instruction retried (now a TLB hit).



When the TLB is full and a new translation must be loaded, an old entry is **evicted**. Replacement policies include LRU, round-robin, or random.

TLB Structure details:

- Typically holds **32 to 1024 entries**
- Lookup is **fully associative** — all entries searched in parallel (unlike a regular cache which uses indexing)
- Each entry may also contain an **ASID (Address Space Identifier)** on some architectures — tags each entry with the process it belongs to, so entries from different processes can coexist without conflict
- On a **context switch**, the TLB must be **flushed** (all entries invalidated) because VPN→PFN mappings belong to a specific process. Systems with ASIDs avoid full flushes — they just check the ASID field on each lookup

Handling TLB Misses — Two Approaches

Aspect	Hardware-Managed TLB	Software-Managed TLB
Used in	Older architectures (CISC, e.g., x86)	Modern architectures (RISC, e.g., MIPS)
On miss	Hardware walks page table automatically	Hardware raises a trap → OS handles it
OS involvement	None	OS looks up page table, inserts translation
After handling	Instruction retried automatically	Instruction retried (must not recurse)
Flexibility	Low (hardware dictates structure)	High (OS can use any page table design)
Hardware complexity	Higher	Lower and simpler

Example: Accessing an Array (TLB Hits and Misses)

Setup: 8-bit virtual address space, page size = 16 bytes → 16 virtual pages (VPN 0–15). Top 4 bits = VPN, bottom 4 bits = offset.

Array: 10 integers × 4 bytes = 40 bytes, starting at virtual address 100.

Address 100 in binary = 01100100 → VPN = 0110 = 6, offset = 0100 = 4.

Array spans three pages:

- VPN 6 → a[0], a[1], a[2]
- VPN 7 → a[3], a[4], a[5], a[6]
- VPN 8 → a[7], a[8], a[9]

Loop: for(i=0; i<10; i++) sum += a[i];

	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 02					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

Access	Element	VPN	TLB State	Result
1	a[0]	6	VPN 6 not in TLB	Miss
2	a[1]	6	VPN 6 now cached	Hit
3	a[2]	6	VPN 6 cached	Hit
4	a[3]	7	VPN 7 not in TLB	Miss
5–7	a[4]–a[6]	7	VPN 7 cached	Hit × 3
8	a[7]	8	VPN 8 not in TLB	Miss
9–10	a[8]–a[9]	8	VPN 8 cached	Hit × 2

Result: 3 misses, 7 hits → TLB hit rate = 70%

This works because of **spatial locality** — elements stored contiguously mean once a page is in the TLB, subsequent accesses on the same page all hit. If the loop runs again immediately, all 10 accesses would be hits (100%) due to **temporal locality**.

Reducing Page Table Size

Since a larger page size means fewer total pages, the page table becomes smaller. For example, quadrupling the page size reduces the page table to **one-quarter** its original size. However, this creates more **internal fragmentation** — programs rarely use every byte of their last page, so larger pages waste more space. Because of this trade-off, most systems use a moderate page size of **4 KB**.

Two main techniques to reduce page table size:

1. Larger Pages — Quadrupling page size cuts the page table to one-quarter but increases internal fragmentation. Simple trade-off.

2. Multi-level Page Tables — Instead of one flat table for all 2^{20} pages, use a two-level structure: an **outer page directory** and **inner page tables**. Only the portions of the address space actually used need inner tables. Unused regions have a null entry in the outer directory — consuming no memory at all.

For a 32-bit address with 4 KB pages:

- 10 bits → outer page directory index
- 10 bits → inner page table index
- 12 bits → offset

This is the approach used in modern x86 systems. It trades a slightly slower lookup (two table accesses instead of one) for dramatically smaller page table memory usage for sparse address spaces.

BEYOND PHYSICAL MEMORY

In practice, the total memory demand of all running processes often exceeds available RAM. Rather than refusing to run these processes, the OS uses **disk storage as an extension of RAM**, enabling the illusion of a much larger memory.

This benefits:

- **Large programs** — can run even if their full size exceeds RAM.
- **Multiprogramming** — many processes can coexist; inactive portions live on disk.
- **Programmers** — no need to manually manage memory overlays as in old systems.

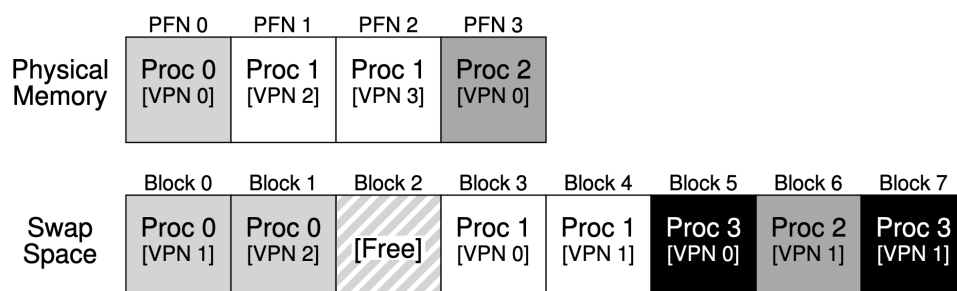
Swap Space

Swap space is a dedicated area on disk (hard drive or SSD) reserved by the OS to store memory pages not currently needed in RAM. Pages can be:

- **Swapped out** — moved from RAM to disk.
- **Swapped in** — moved back from disk to RAM.

Both happen in **page-sized units**.

The OS maintains a **present bit** in each PTE to track whether a page is in RAM (present bit = 1) or on disk in swap space (present bit = 0).



Physical Memory and Swap Space

Page Fault and Its Control Flow

A **page fault** occurs when a process accesses a virtual page whose **present bit** = 0 in the page table, meaning the page is not currently in physical memory. The hardware detects this and raises a trap to the OS.

Complete Page Fault Handling Sequence

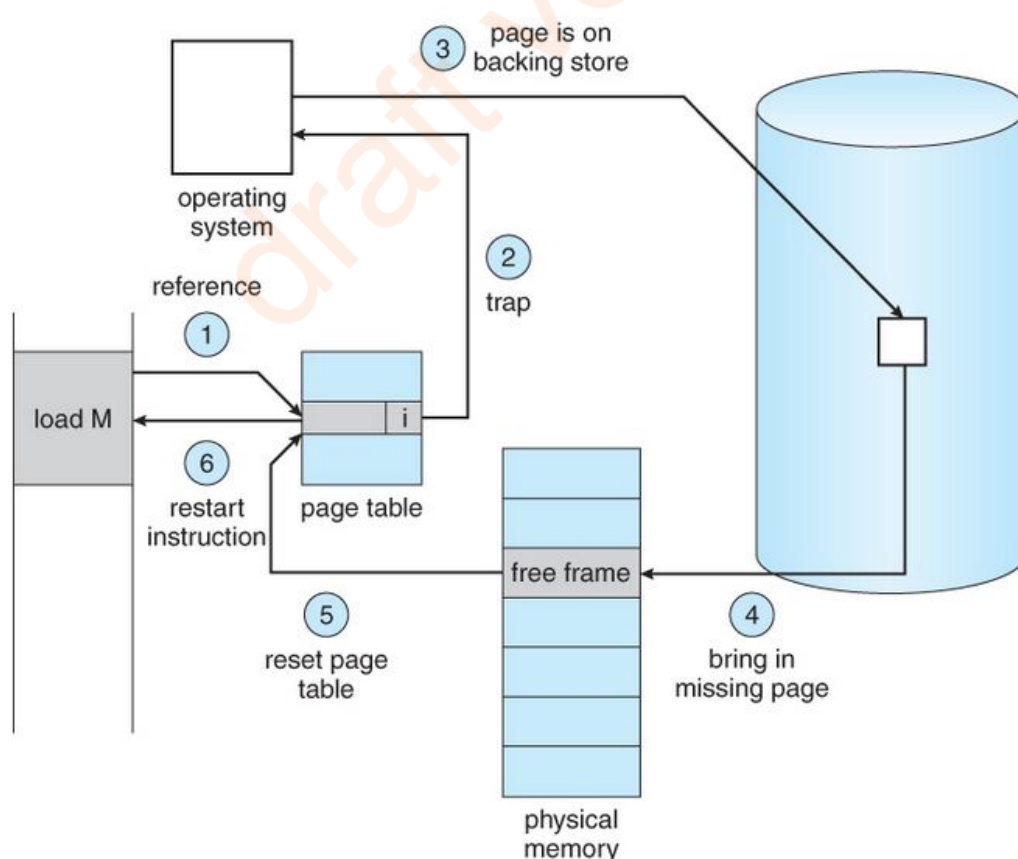
Step 1: CPU generates virtual address → hardware extracts VPN → checks TLB.

Step 2 (TLB Miss): Hardware checks page table → reads PTE.

- If present bit = 1 → page is in RAM → load PFN into TLB → retry → TLB hit → success.
- If present bit = 0 → **Page Fault triggered**.

Step 3 — Page Fault Handling by OS:

1. Check the process's internal table (in PCB) — is this a **valid** address?
 - If **invalid** → terminate the process (segmentation fault).
 - If **valid but not in RAM** → proceed to bring the page in.
2. Find a **free frame** in physical memory.
 - If free frame available → use it directly.
 - If no free frame → run a **page replacement policy** to evict a victim page (write dirty victim to disk first if dirty bit = 1).
3. Schedule a **disk I/O** to load the required page from swap into the free frame.
4. While waiting for disk I/O (which is slow), the OS **blocks the faulting process** and runs another process — improving CPU utilization.
5. When disk I/O completes → update PTE (set present bit = 1, record new PFN), update internal tables.
6. **Restart the faulting instruction** — the process resumes as if the page was always in memory.

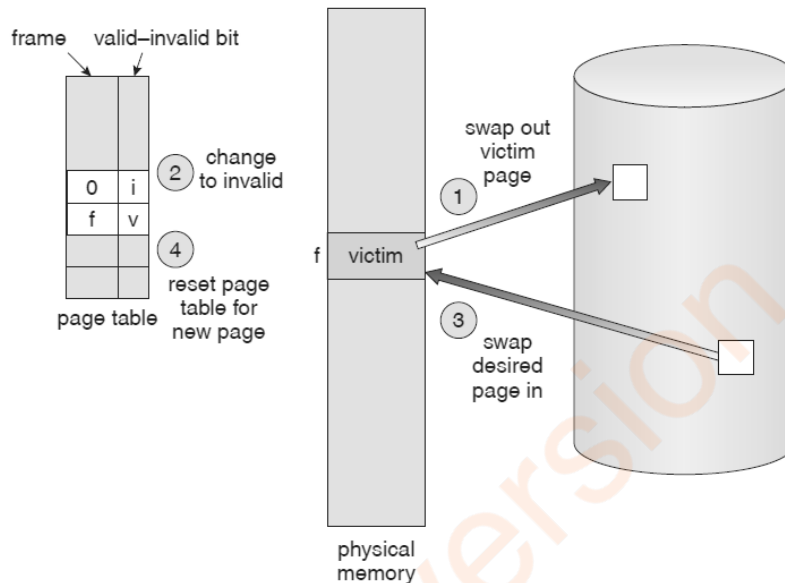


Page Replacement Policies

When memory is full and a new page must be loaded, the OS must evict an existing page. The algorithm used to choose the **victim page** is called the **page replacement policy**. The goal is to **minimize page faults**.

General page replacement steps:

1. Find the required page's location on disk.
2. Find a free frame — if none, select a victim frame using the replacement algorithm.
3. Write victim to disk (only if dirty bit = 1). Update page and frame tables.
4. Load the desired page into the freed frame. Update page and frame tables.
5. Restart the interrupted instruction.



1. FIFO (First-In, First-Out)

The **oldest page** in memory (loaded first) is evicted first. Pages maintained in a queue; front of the queue is the next victim.

Strength: Very simple to implement. **Weakness:** May evict frequently-used pages — age does not equal importance.

Belady's Anomaly: In FIFO, increasing the number of frames can **increase** page faults for certain reference strings. This counter-intuitive behavior is **unique to FIFO**.

Example: Reference string 1 2 3 4 1 2 5 1 2 3 4 5

- With 3 frames → 9 faults
- With 4 frames → 10 faults ← more frames, more faults!

2. Optimal Page Replacement (OPT)

Definition: Replace the page that will **not be used for the longest period of time in the future**.

Gives the **lowest possible page fault rate** of all algorithms. Never suffers from Belady's anomaly. Serves as a **theoretical benchmark** for comparing other algorithms.

Weakness: Requires future knowledge of the reference string — **not implementable in practice**. Used only as a theoretical reference.

3. LRU (Least Recently Used)

Definition: Replace the page that has **not been used for the longest time in the past** — the one with the oldest last-use time.

LRU is based on **temporal locality**: pages used recently are likely to be used again soon. It looks **backward** in time (past usage) rather than forward (like OPT). Like OPT, LRU **does not suffer from Belady's anomaly** — both are called **stack algorithms**.

A **stack algorithm** is one where the set of pages in memory with n frames is always a subset of the set with $n+1$ frames. This guarantees adding more frames **never increases** page faults.

Hardware Support for LRU:

Counter method: Each PTE has a time-of-use field. A hardware clock increments on every memory reference. When a page is accessed, the clock value is copied into its PTE. The page with the **smallest (oldest)** time value is evicted. Requires searching all PTEs on replacement.

Stack method: A doubly linked list of page numbers is maintained. When a page is referenced, it is moved to the top. Most recently used is always at top; least recently used is always at bottom. No search needed — just remove from bottom.

4. Clock Page Replacement Algorithm (Modified Clock)

Also called the **Second Chance** algorithm. Uses a **circular list** of page frames with a **Reference bit (R)** and **Modified/Dirty bit (M)** per page. A clock hand sweeps the circle.

Basic Clock: When replacement is needed, examine the page the hand points to:

- If $R = 0 \rightarrow$ evict it.
- If $R = 1 \rightarrow$ set $R = 0$, advance the hand (give it a second chance).
- Continue until a page with $R = 0$ is found.

Modified Clock: Considers both R and M . Prefers clean pages ($M = 0$) to avoid unnecessary disk writes. Eviction priority — **best victim first**:

Class	R	M	Action
1st choice	0	0	Evict immediately — not used, not modified
2nd choice	0	1	Evict but write to disk first — not used, but dirty
3rd choice	1	0	Clear R, give second chance — recently used, clean
4th choice	1	1	Clear R, give second chance — worst victim

The hand makes passes until a victim is found. This is a practical, efficient approximation of LRU used in **real operating systems**.

Comparison of Page Replacement Algorithms

Algorithm	Basis for Eviction	Belady's Anomaly	Practical?	Performance
FIFO	Oldest loaded page	Yes	Yes	Poor
Optimal (OPT)	Longest future non-use	No	No (needs future)	Best possible
LRU	Longest past non-use	No	Yes (with hardware)	Good
Clock	R bit (approximates LRU)	No	Yes (efficient)	Good approximation

QUESTION — Comprehensive Page Replacement

A system has 3 page frames. Consider the following page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
For the above reference string:

1. Apply FIFO page replacement algorithm and find:
 - Frame contents after each reference
 - Number of page faults
2. Apply LRU page replacement algorithm and find:
 - Frame contents after each reference
 - Number of page faults
3. Apply Optimal page replacement algorithm and find:
 - Frame contents after each reference
 - Number of page faults
4. Which algorithm gives minimum page faults and why?
5. Explain whether FIFO may suffer from Belady's Anomaly.
6. A page table contains the following entries:

Page	R	M
0	0	0
1	0	1
2	1	0
3	1	1

Which page class is preferred first in Modified Clock algorithm?

FULL STUDY ANSWER

1. FIFO Page Replacement

Rule: Oldest loaded page removed first.

FIFO Table

Ref	Frames	Fault?
7	7 - -	Yes
0	7 0 -	Yes
1	7 0 1	Yes
2	2 0 1	Yes
0	2 0 1	No
3	2 3 1	Yes
0	2 3 0	Yes
4	4 3 0	Yes
2	4 2 0	Yes
3	4 2 3	Yes
0	0 2 3	Yes
3	0 2 3	No
2	0 2 3	No

Total faults = 10

2. LRU Page Replacement

Rule: Remove least recently used page.

LRU Table

Ref	Frames	Fault?
7	7 - -	Yes
0	7 0 -	Yes
1	7 0 1	Yes
2	2 0 1	Yes
0	2 0 1	No
3	2 0 3	Yes
0	2 0 3	No
4	4 0 3	Yes
2	4 0 2	Yes
3	4 3 2	Yes
0	0 3 2	Yes
3	0 3 2	No
2	0 3 2	No

Total faults = 9

3. Optimal Page Replacement

Rule: Replace page that will not be used for longest future time.

Optimal Table

Ref	Frames	Fault?
7	7 - -	Yes
0	7 0 -	Yes
1	7 0 1	Yes
2	2 0 1	Yes
0	2 0 1	No
3	2 0 3	Yes
0	2 0 3	No
4	2 4 3	Yes
2	2 4 3	No
3	2 4 3	No
0	2 0 3	Yes
3	2 0 3	No
2	2 0 3	No

Total faults = 7

4. Which Algorithm is Best?

Optimal gives minimum page faults.

Reason:

- It knows future references.
- Replaces page used farthest in future.

But:

- Impossible in real systems because future references are unknown.

So real OS uses approximations like:

- LRU
- Clock
- Second Chance

5. Belady's Anomaly

Definition: Increasing number of frames may increase page faults.

Occurs mainly in:

- FIFO

Does NOT occur in:

- LRU
- Optimal

Because:

- LRU and Optimal satisfy stack property.

6. Modified Clock Algorithm

Priority order:

1. (0,0)
2. (0,1)
3. (1,0)
4. (1,1)

Meaning: Prefer unreferenced and clean pages first.

Given table:

Page	R	M
0	0	0
1	0	1
2	1	0
3	1	1

Best class = (0,0)

Therefore: Page 0 is selected first.

Thrashing

Thrashing is a condition where the OS spends **more time swapping pages** between RAM and disk than executing actual process instructions, resulting in a severe drop in CPU utilization and overall system performance.

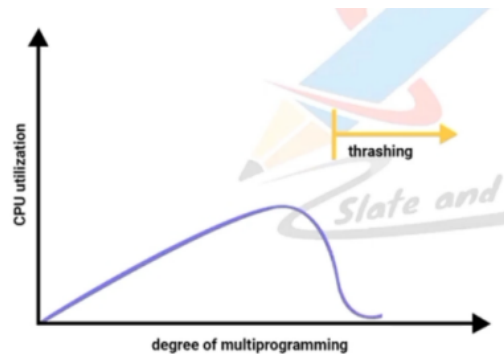
Why Thrashing Occurs

The OS scheduler observes low CPU utilization and loads more processes to improve it. But more processes mean **fewer frames per process**. Each process lacks the minimum frames needed to hold its working pages, causing frequent page faults. Every page fault triggers disk I/O, during which the process

waits. The CPU switches to other processes, which themselves fault immediately. The result: the system is always doing I/O, never doing useful work.

The cycle:

Low CPU utilization → OS adds more processes → each process gets fewer frames → more page faults → more swapping → even lower CPU utilization → loop continues



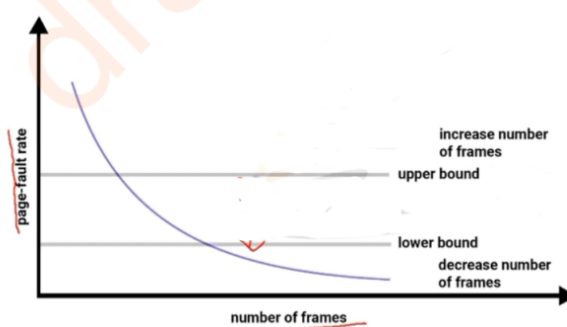
The **sweet spot** is at the peak — maximum utilization before thrashing begins. To the right of the peak, performance collapses as the system drowns in page faults.

How the OS Handles Thrashing

1. Working Set / Admission Control (Older approach): The OS defines each process's **working set** — the set of pages it actively uses. Only processes whose combined working sets fit in physical memory are allowed to run simultaneously. Others are suspended (swapped out entirely) until memory is available. This approach does less work but does it efficiently.

2. Page Fault Rate Control: The OS monitors each process's page fault rate and maintains it within acceptable bounds:

- If fault rate **exceeds upper bound** → process needs more frames → allocate more.
- If fault rate **falls below lower bound** → process has too many frames → reclaim some.



3. Out-of-Memory (OOM) Killer (Linux/Modern Systems): When memory is critically oversubscribed, the Linux kernel's OOM killer selects and **terminates** the most memory-intensive process to immediately free up RAM. This is a last resort and can be problematic if a critical process (like the display server) is killed.

MODEL QUESTIONS

1. On a system using demand-paged memory, it takes 120ns to satisfy a memory request if the page is in memory. If the page is not in memory, the request takes 5ms. What page fault rate is needed to achieve an effective access time of $1\mu\text{s}$? (3 MARKS)
2. A computer has four page frames. The time of loading (TLD), time of last access (TLA), and the R (reference) and M (modified) bits for each page are as shown below (the times are logical clock time values): If a page is to be replaced, what page will be replaced by the FIFO, LRU and modified clock algorithms? (3 MARKS)

Page	TLD	TLA	R	M
0	126	280	1	0
1	230	265	0	1
2	240	270	1	0
3	110	285	1	1

3. Translate the logical address 0001010010111010 into its corresponding physical address under a segmentation system with a maximum segment size of 1024, using a segment table where the base address of each segment is determined as: $22 + 4,096 \times \text{segment number}$. (5 MARKS)
4. A computer provides each process with 65,536 bytes of address space divided into pages of 4096 bytes each. A particular process has a text size of 32,768 bytes, a data size of 16,386 bytes, and a stack size of 15,870 bytes. Will this process fit in the machine's address space? Suppose that instead of 4096 bytes, the page size was 512 bytes, would it then fit? Each page must contain either text, data, or stack, not a mixture of two or three of them. (4 MARKS)
5. Consider the following C program:

```
int X[N], step = M;  
for (int i = 0; i < N; i += step)  
    X[i] = X[i] + 1;
```


If this program is run on a machine with a 4-KB page size and 64-entry TLB, what values of M and N will cause a TLB miss for every execution of the inner loop? (3 MARKS)
6. Consider the following page reference string: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6. With a frame allocation of 4, determine the number of page faults for the FIFO, optimal and LRU page replacement policies. (6 MARKS)

Note: from address translation topics onwards the model questions are not solved along with topics – so do that. Sorry for unexpected things.